

Quicksort explanation in C++

1. explain from Quicksort algorithm in c++: void quicksort(vector<int> &arr, int low , int high) { if(low<high) { int pivotPos = partitionArray(arr,low,high); quicksort(arr,low,pivotPos-1); quicksort(arr,pivotPos+1,high); } }.2
2. explain: int partitionArray(vector<int> &arr, int low , int high) { int pivot = arr[high]; int pos = low-1; for(int index = low;index<high;index++) { if(arr[index]<pivot) { pos++; swap(arr[index],arr[pos]); } } pos++; swap(arr[high],arr[pos]); return pos; }. 5



1. explain from Quicksort algorithm in c++:

```
void quicksort(vector<int> &arr, int low , int high)
{
    if(low<high)
    {
        int pivotPos = partitionArray(arr,low,high);
        quicksort(arr,low,pivotPos-1);
        quicksort(arr,pivotPos+1,high);
    }
}
```

Let's break down your **Quicksort** function line by line so you clearly understand how it works:

Concept:

Quicksort is a **divide-and-conquer** sorting algorithm.

It works by:

1. Choosing a **pivot** element from the array.
2. **Partitioning** the array so that:
 - All elements **less than pivot** go to the **left**.
 - All elements **greater than pivot** go to the **right**.
3. Then it **recursively** sorts both left and right parts.

Code Explanation:

cpp

```
void quicksort(vector<int> &arr, int low , int high)
{
```

- **vector<int> &arr**: The array (vector) to be sorted, passed **by reference** so changes affect the original vector.
- **low** and **high**: The **starting** and **ending indices** of the portion of the array currently being sorted.

cpp

```
if(low < high)
```

- This is the **base condition** for recursion.
- If `low >= high`, it means the array (or subarray) has **0 or 1 elements**, which is already sorted.
- So, sorting is only done when there are **at least 2 elements**.

cpp

```
int pivotPos = partitionArray(arr, low, high);
```

- `partitionArray()` is a **helper function** (defined elsewhere) that:
 - Chooses a **pivot** element (commonly `arr[high]`).
 - Rearranges the array so that:
 - Elements smaller than pivot are on the left,
 - Elements greater are on the right.
 - Returns the **final index (pivotPos)** of the pivot after partitioning.

cpp

```
quicksort(arr, low, pivotPos - 1);
quicksort(arr, pivotPos + 1, high);
```

- These two recursive calls sort the **left** and **right** parts separately:
 - `low → pivotPos - 1` → elements smaller than pivot
 - `pivotPos + 1 → high` → elements larger than pivot

Example Flow

Suppose:

cpp

```
arr = [10, 7, 8, 9, 1, 5];
```

```
low = 0, high = 5
```

1. `partitionArray()` picks 5 as pivot.
→ After partition: [1, 5, 8, 9, 10, 7]
→ Returns `pivotPos = 1`.
2. Then recursively sorts:
 - Left side: [1] (already sorted)
 - Right side: [8, 9, 10, 7]
3. Keeps repeating until all subarrays are size 1.

Visualization:

Step	Subarray Range	Pivot	Action	Result
1	[0,5]	5	Partition	[1,5,8,9,10,7]
2	[0,0]	—	Done	[1]
3	[2,5]	7	Partition	[1,5,7,8,9,10]
...	...	...	...	Sorted

Summary:

- `if(low<high)` → stops recursion.
- `partitionArray()` → organizes elements around pivot.
- **Recursive calls** → sort smaller subarrays until all are sorted.

Would you like me to show the `partitionArray()` **function** that goes with this `quicksort` code too (so you can see how pivoting works)?



2. explain:

```
int partitionArray(vector<int> &arr, int low , int high)
{
    int pivot = arr[high];
    int pos = low-1;
    for(int index = low; index<high; index++)
    {
        if(arr[index]<pivot)
        {
            pos++;
            swap(arr[index],arr[pos]);
        }
    }
    pos++;
    swap(arr[high],arr[pos]);
    return pos;
}
```

Excellent — this is the `partitionArray()` function that works hand-in-hand with your `quicksort()` function.

Let's go through it **line by line** so you fully understand what's happening 🙌

Purpose of partitionArray

This function takes an array segment (**low** to **high**) and:

- Selects one element as the **pivot** (usually the last element).
- **Rearranges** the elements so that:
 - All elements **smaller than pivot** come **before** it.
 - All elements **greater than pivot** come **after** it.
- Finally, it returns the **final (correct) index** of the pivot after rearrangement.

Code Breakdown

cpp

```
int pivot = arr[high];
```

- The **pivot element** is chosen as the **last element** in the range [**low**, **high**].
- For example, if **arr** = [10, 7, 8, 9, 1, 5] and **low**=0, **high**=5, then **pivot** = 5.

cpp

```
int pos = low - 1;
```

- **pos** keeps track of the **boundary** between elements smaller than the pivot and the rest.
- Initially, there are **no** smaller elements, so **pos** is set **before** the first index (**low** - 1).

cpp

```
for (int index = low; index < high; index++)
{
    if (arr[index] < pivot)
    {
        pos++;
        swap(arr[index], arr[pos]);
    }
}
```

Let's break this loop:

- It scans through all elements **except the pivot** (**index** < **high**).
- Whenever an element **smaller than the pivot** is found:
 - Move the boundary **pos** one step right (**pos++**).
 - **Swap** that element with the element at **pos**, bringing it to the "smaller elements" side.

So after this loop:

- **arr[low ... pos]** contains elements **smaller than pivot**.
- **arr[pos+1 ... high-1]** contains elements **greater or equal to pivot**.

cpp

```
pos++;
swap(arr[high], arr[pos]);
```

- Move **pos** one step forward — this is where the pivot should be placed.
- Swap the **pivot** with the element at **pos**.
- Now:
 - All elements **left** of pivot are smaller.
 - All elements **right** of pivot are larger or equal.
- The pivot is now in its **correct sorted position**.

cpp

```
return pos;
```

- Return the pivot's final index — used by `quicksort()` to divide the array into left and right subarrays.

Example Walkthrough

Suppose:

cpp

```
arr = [10, 7, 8, 9, 1, 5];
low = 0;
high = 5;
pivot = 5;
```

Step	index	arr[index]	arr[index] < pivot?	pos	Action	Array after step
init	—	—	—	-1	—	[10, 7, 8, 9, 1, 5]
0	0	10	✗	-1	no swap	[10, 7, 8, 9, 1, 5]
1	1	7	✗	-1	no swap	[10, 7, 8, 9, 1, 5]
2	2	8	✗	-1	no swap	[10, 7, 8, 9, 1, 5]
3	3	9	✗	-1	no swap	[10, 7, 8, 9, 1, 5]
4	4	1	✓	0	swap(1,10)	[1, 7, 8, 9, 10, 5]

Now **pos** = 0.

After loop:

```
cpp

pos++; // pos = 1
swap(arr[high], arr[pos]); // swap(5,7)
```

Final array:

```
csharp

[1, 5, 8, 9, 10, 7]
```

Return `pos = 1`.

So pivot (5) is at its correct position.

 Summary

Line	Purpose
<code>int pivot = arr[high];</code>	Choose last element as pivot
<code>int pos = low - 1;</code>	Track boundary for smaller elements
<code>for(...)</code>	Move smaller elements to left side
<code>swap(arr[high], arr[pos+1]);</code>	Place pivot in correct position
<code>return pos;</code>	Return pivot's final index

Would you like me to show a **visual trace of quicksort() and partitionArray() together** on a short array (e.g. [4, 2, 7, 1, 3]) so you can see the recursive flow step by step?